

Salesforce Project Documentation

WhatNext Vision Motors CRM

Implementation Report Based on Completed Work

Prepared By	Rithik Mishra
Environment	Salesforce Developer Edition
Project	WhatNext Vision Motors
Date	September 10, 2026

Scope of this report: only the Salesforce components actually configured, activated, tested, or scheduled during the project work.

Contents

Section	Title
1	Project Overview
2	Implemented Objectives and Scope
3	Data Model: Objects, Fields and Relationships
4	Lightning App and Custom Tabs
5	Declarative Automation - Record-Triggered Flows
5.1	Auto Assign Dealer
5.2	Test Drive Reminder
6	Programmatic Automation - Apex Trigger and Handler
7	Asynchronous Apex - Batch and Scheduler
8	Implementation Testing and Evidence
9	Final Implemented Architecture
10	Conclusion

1 Project Overview

WhatNext Vision Motors is implemented as a custom Salesforce CRM for managing vehicles, dealers, customers, vehicle orders, test drives, and service requests. The implementation combines Salesforce configuration, Record-Triggered Flows, Apex Triggers, Batch Apex, and Scheduled Apex to automate important automotive sales and service processes.

The completed solution focuses on four practical requirements: maintaining vehicle and dealer data, assigning a matching dealer to pending orders based on customer location, sending a test-drive reminder email one day before the scheduled date, and enforcing vehicle-stock rules during order processing.

2 Implemented Objectives and Scope

- Create six project-specific custom objects and define their business fields.
- Configure Lookup Relationships among Vehicle, Dealer, Customer, Order, Test Drive, and Service Request.
- Create custom tabs and a dedicated Lightning app named WhatNext Vision Motors.
- Create and activate an Auto Assign Dealer Record-Triggered Flow.
- Create and activate a Test Drive Reminder flow using a scheduled path and Send Email action.
- Create an Apex Trigger Handler and Trigger to block confirmed orders when vehicle stock is unavailable and to update stock for confirmed orders.
- Create Batch Apex and a Schedulable class for processing pending orders after stock replenishment.
- Schedule the batch processing job to execute daily at midnight.

Items not performed in the work documented here - such as custom security profiles, role hierarchy, data migration rules, Dynamic Forms, Change Sets, and Apex test classes - are intentionally excluded from this report.

3 Data Model: Objects, Fields and Relationships

The project uses custom objects and Lookup Relationships. The record-name fields created during object creation are used as the primary display names, while additional custom fields store business data.

Object	Implemented Fields	Implemented Relationships
Vehicle__c	Vehicle Name (record name); Vehicle Model; Stock Quantity; Price; Status	Lookup to Vehicle Dealer
Vehicle_Dealer__c	Dealer Name (record name); Dealer Location; Dealer Code; Phone; Email	Referenced by Vehicle and Vehicle Order
Vehicle_Customer__c	Customer Name (record name); Email; Phone; Address; Preferred Vehicle Type	Referenced by Order, Test Drive, Service Request
Vehicle_Order__c	Order Name (Auto Number); Order Date; Status	Lookup to Vehicle Customer, Vehicle, and Vehicle Dealer
Vehicle_Test_Drive__c	Test Drive Name (Auto Number); Test Drive Date; Status	Lookup to Vehicle Customer and Vehicle
Vehicle_Service_Request__c	Service Request Name (Auto Number); Service Date; Issue Description; Status	Lookup to Vehicle Customer and Vehicle

3.1 Picklist and Auto-Number Configuration

Configuration	Values / Format
Vehicle Model	Sedan, SUV, EV
Vehicle Status	Available, Out of Stock, Discontinued
Order Status	Pending, Confirmed, Delivered, Canceled
Test Drive Status	Scheduled, Completed, Canceled
Service Request Status	Requested, In Progress, Completed

Order Auto Number	ORD-{0000}
Test Drive Auto Number	TD-{0000}
Service Request Auto Number	SR-{0000}
Dealer Code Auto Number	DLR-{0000}

The screenshot displays the Salesforce Setup interface for Custom Tabs. The left sidebar shows the navigation menu with 'Setup' and 'Object Manager' options. The main content area is titled 'Custom Tabs' and includes a 'Help for this Page' link. Below the title, there is a table of Custom Object Tabs.

Action	Label	Tab Style	Description
Edit Del	Vehicle Customers	Dice	
Edit Del	Vehicle Dealers	Headset	
Edit Del	Vehicle Orders	Leaf	
Edit Del	Vehicles	Helicopter	
Edit Del	Vehicle Service Requests	Computer	
Edit Del	Vehicle Test Drives	Square	

Figure 1: Custom Object Tabs created for the six project objects.

4 Lightning App and Custom Tabs

A dedicated Lightning application named WhatNext Vision Motors was created through App Manager. The application acts as the central workspace for the custom CRM records. The navigation includes Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives, Vehicles, and additional Salesforce items such as Reports and Dashboards where needed.

Custom Object Tabs were created for all six custom objects so users can access records directly from the application's navigation bar.

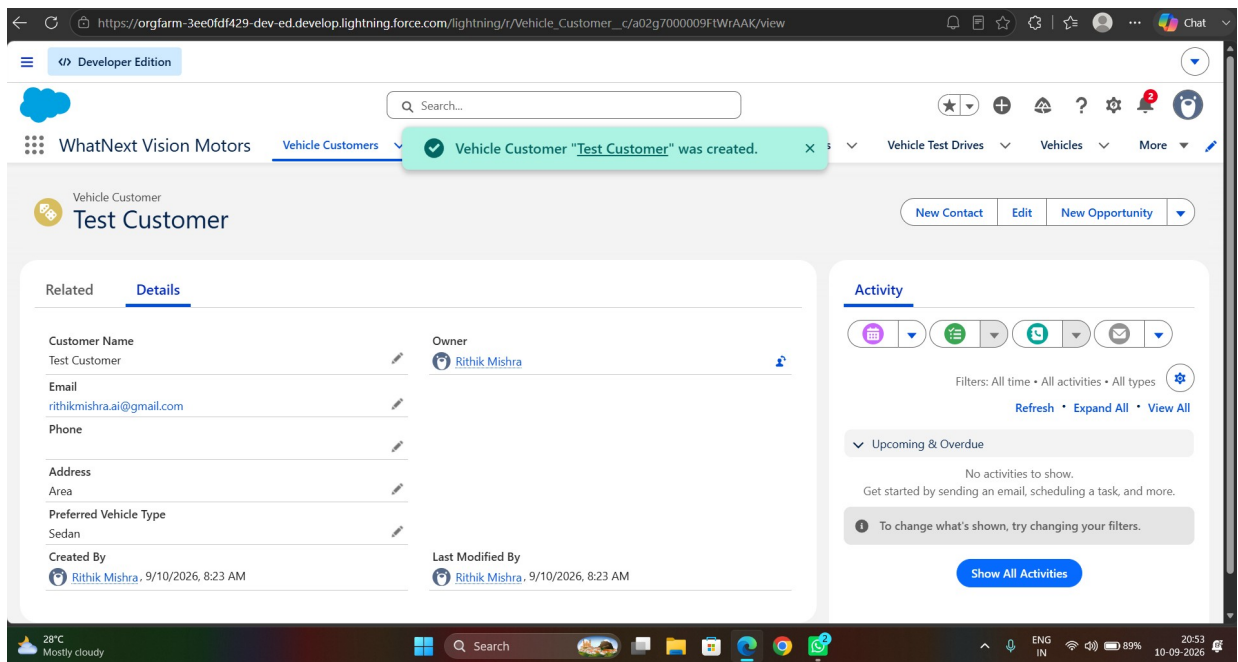


Figure 2: WhatNext Vision Motors Lightning app showing project navigation and a Vehicle Customer record.

5 Declarative Automation - Record-Triggered Flows

Two Record-Triggered Flows were created and activated. The first performs location-based dealer assignment for pending vehicle orders. The second sends an email reminder before a scheduled test drive.

5.1 Auto Assign Dealer Flow

The Auto Assign Dealer flow is triggered when a Vehicle Order record is created with Status = Pending. It retrieves the related Vehicle Customer, reads the customer's Address, finds a Vehicle Dealer whose Dealer Location matches that address, and updates the triggering Vehicle Order's Dealer lookup with the matched dealer's Record ID.

Setting	Implemented Configuration
Trigger Object	Vehicle Order
Trigger	A record is created
Entry Condition	Status Equals Pending
Optimization	Actions and Related Records
Element 1	Get Customer Information - Record ID equals triggering order's Vehicle Customer Record ID
Element 2	Get Nearest Dealer - Dealer Location equals Get Customer Information > Address
Element 3	Assign Dealer to Order - update triggering Vehicle Order; Dealer = Get

	Nearest Dealer > Record ID
Status	Activated

Technical note: the implementation uses exact text matching between customer Address and dealer Dealer Location. Therefore, the flow label says 'Nearest Dealer', but the configured logic is a location match rather than a geographic distance calculation.

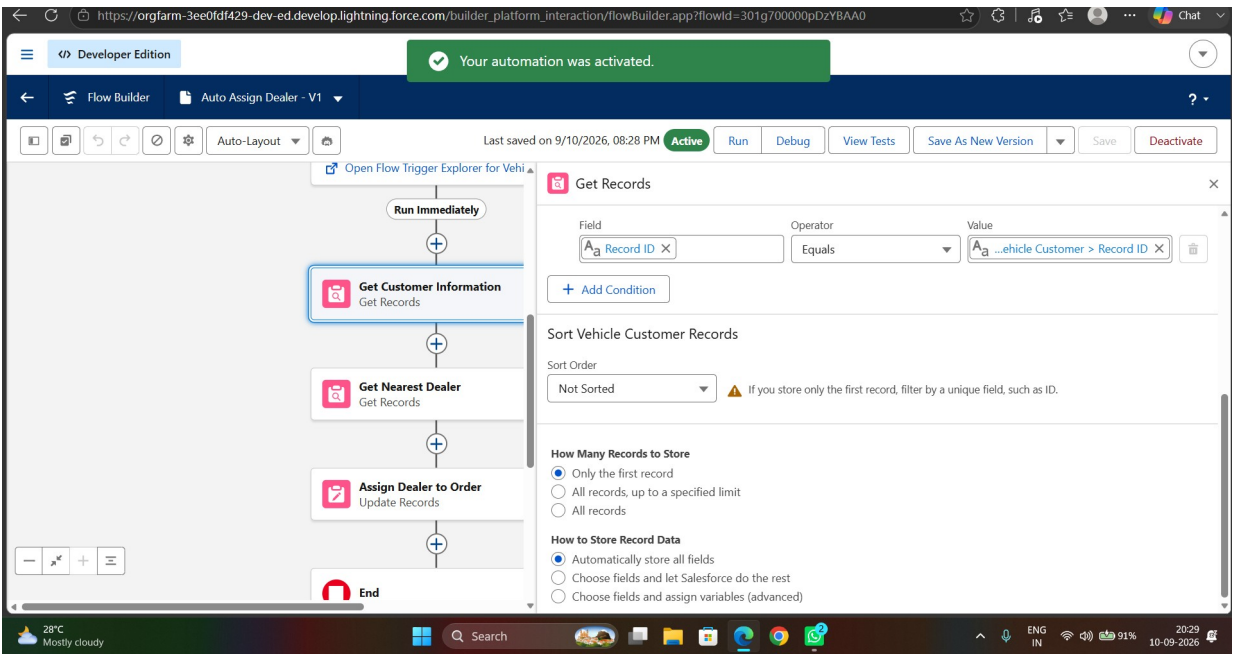


Figure 3: Activated Auto Assign Dealer flow in Flow Builder.

5.2 Test Drive Reminder Flow

The Test Drive Reminder flow is triggered when a Vehicle Test Drive record is created or updated with Status = Scheduled. A scheduled path named Reminder Before Test Drive runs one day before the Test Drive Date. The scheduled path retrieves the related customer and sends an email to the customer's Email field.

Setting	Implemented Configuration
Trigger Object	Vehicle Test Drive
Trigger	A record is created or updated
Entry Condition	Status Equals Scheduled
Scheduled Path	Reminder Before Test Drive
Time Source	Test Drive Date
Offset	1 Day Before
Get Records	Get Customer Information
Recipient	Get Customer Information > Email
Subject	Reminder: Your Test Drive is Tomorrow!
Status	Activated and functionally verified

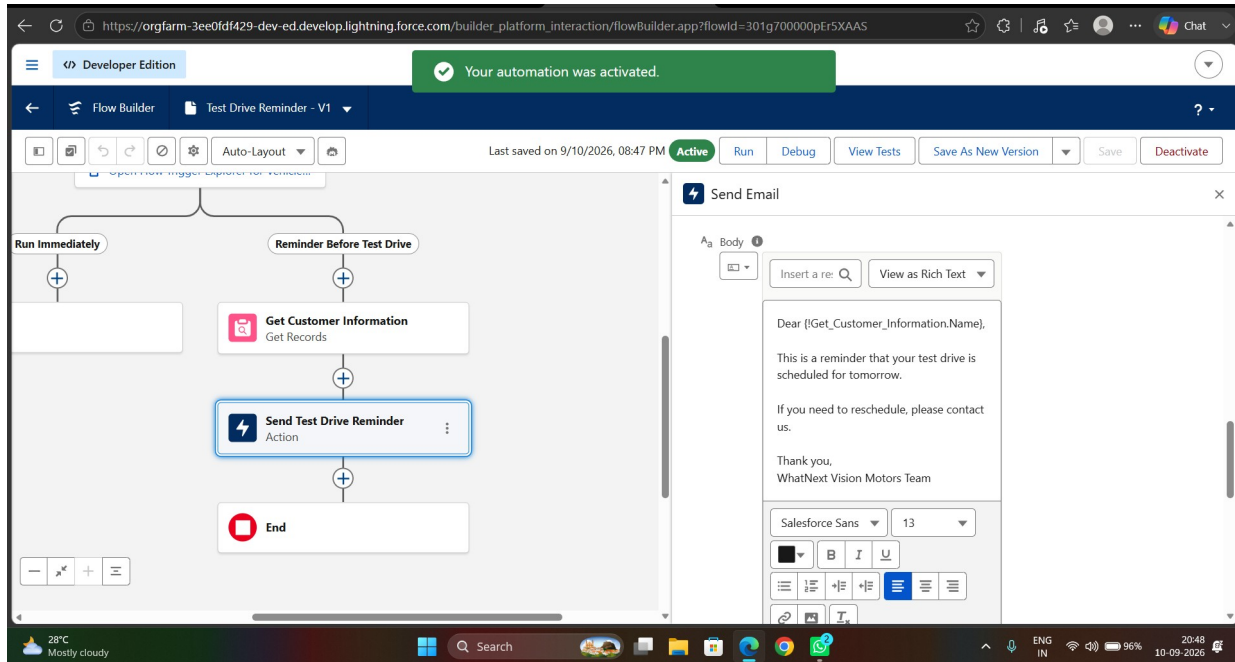


Figure 4: Activated Test Drive Reminder flow with scheduled-path actions.

5.3 Email Reminder Verification

A test Vehicle Customer and Vehicle Test Drive were created. The test drive record TD-0001 was configured for September 11, 2026 with Status = Scheduled. The flow retrieved the customer email and the reminder message was received successfully.

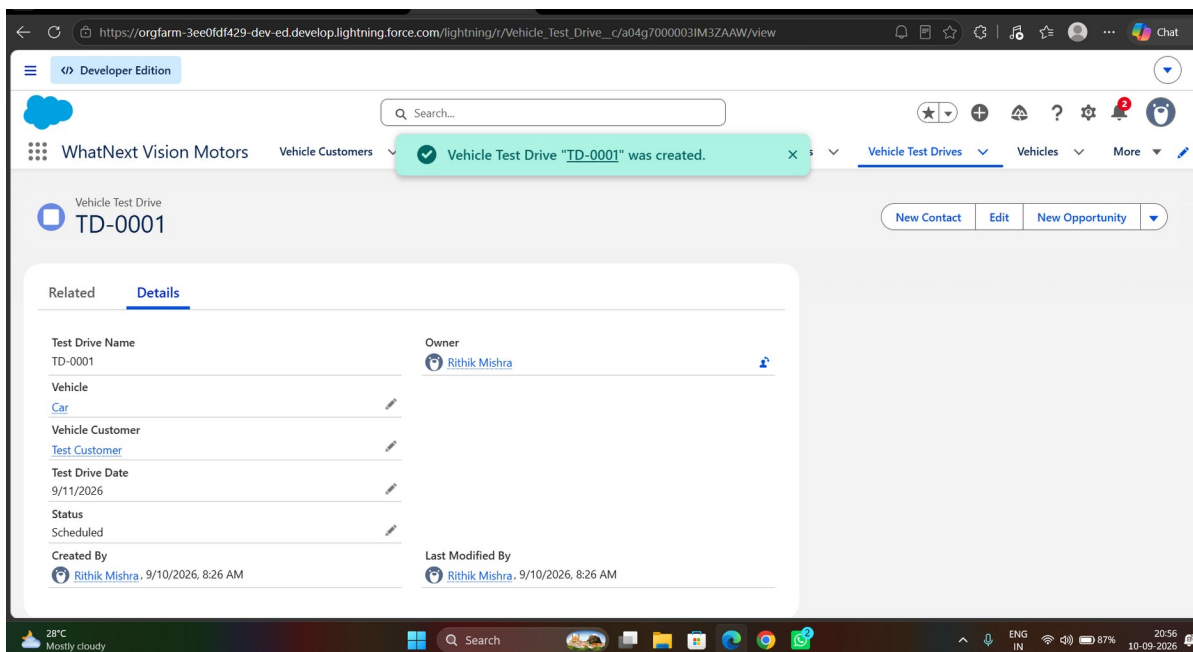


Figure 5: Test Drive record TD-0001 created with Scheduled status.

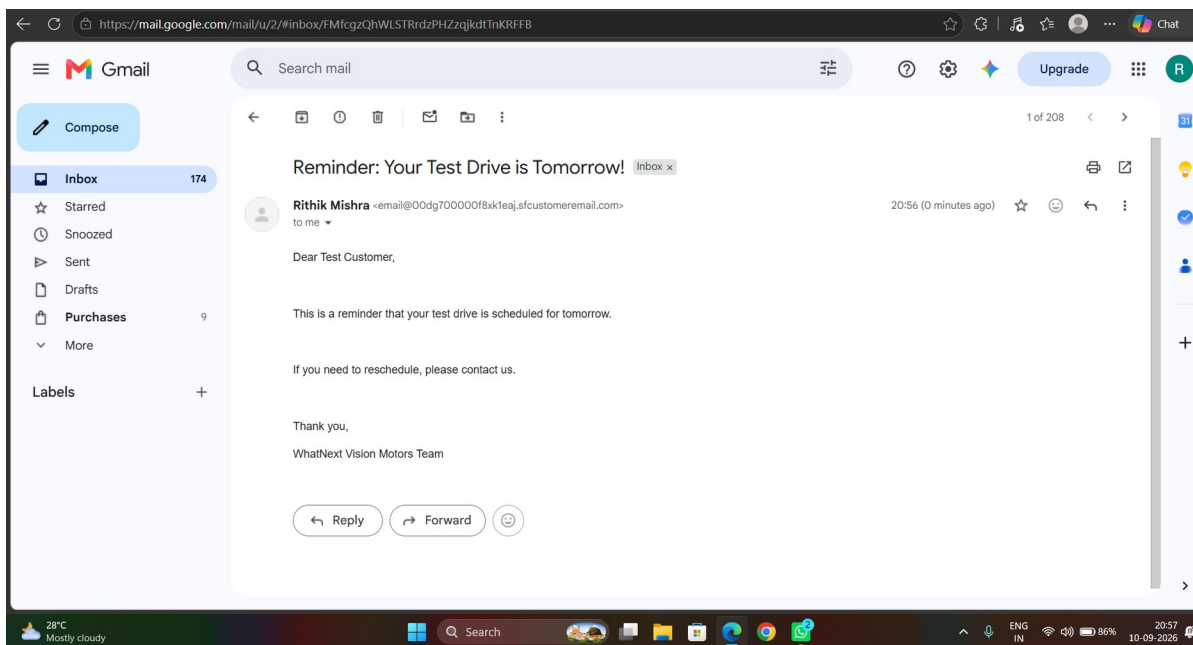


Figure 6: Test-drive reminder email received successfully.

6 Programmatic Automation - Apex Trigger and Handler

The order stock-validation requirement was implemented using a Trigger Handler pattern. VehicleOrderTrigger delegates logic to VehicleOrderTriggerHandler for before and after DML contexts.

The implemented logic blocks a Confirmed order when the selected vehicle has zero or negative stock. When an order becomes Confirmed and stock is available, the handler reduces the vehicle Stock Quantity.

6.1 VehicleOrderTriggerHandler

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(
        List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldOrders,
        Boolean isBefore,
        Boolean isAfter,
        Boolean isInsert,
        Boolean isUpdate
    ) {
        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders);
        }

        if (isAfter && (isInsert || isUpdate)) {
            updateStockOnOrderPlacement(newOrders, oldOrders, isInsert, isUpdate);
        }
    }

    private static void preventOrderIfOutOfStock(
        List<Vehicle_Order__c> orders
    ) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c orderRec : orders) {
            if (orderRec.Vehicle__c != null &&
                orderRec.Status__c == 'Confirmed') {
                vehicleIds.add(orderRec.Vehicle__c);
            }
        }

        if (vehicleIds.isEmpty()) return;

        Map<Id, Vehicle__c> vehicleStockMap =
            new Map<Id, Vehicle__c>({
                SELECT Id, Stock_Quantity__c
                FROM Vehicle__c
                WHERE Id IN :vehicleIds
            });

        for (Vehicle_Order__c orderRec : orders) {
            Vehicle__c vehicle = vehicleStockMap.get(orderRec.Vehicle__c);

            if (vehicle != null &&
                (vehicle.Stock_Quantity__c == null ||
                 vehicle.Stock_Quantity__c <= 0)) {
                orderRec.addError(
                    'This vehicle is out of stock. Order cannot be placed.'
                );
            }
        }
    }
}
```

```

    }
}

private static void updateStockOnOrderPlacement(
    List<Vehicle_Order__c> orders,
    Map<Id, Vehicle_Order__c> oldOrders,
    Boolean isInsert,
    Boolean isUpdate
){
    Set<Id> vehicleIds = new Set<Id>();

    for (Vehicle_Order__c orderRec : orders) {
        Boolean newlyConfirmed = false;

        if (isInsert) {
            newlyConfirmed = orderRec.Status__c == 'Confirmed';
        } else if (isUpdate) {
            Vehicle_Order__c oldOrder = oldOrders.get(orderRec.Id);
            newlyConfirmed =
                orderRec.Status__c == 'Confirmed' &&
                oldOrder.Status__c != 'Confirmed';
        }

        if (newlyConfirmed && orderRec.Vehicle__c != null) {
            vehicleIds.add(orderRec.Vehicle__c);
        }
    }

    if (vehicleIds.isEmpty()) return;

    Map<Id, Vehicle__c> vehicleStockMap =
        new Map<Id, Vehicle__c>([
            SELECT Id, Stock_Quantity__c
            FROM Vehicle__c
            WHERE Id IN :vehicleIds
            FOR UPDATE
        ]);

    Map<Id, Integer> quantityRequired = new Map<Id, Integer>();

    for (Vehicle_Order__c orderRec : orders) {
        Boolean newlyConfirmed = false;

        if (isInsert) {
            newlyConfirmed = orderRec.Status__c == 'Confirmed';
        } else if (isUpdate) {
            Vehicle_Order__c oldOrder = oldOrders.get(orderRec.Id);
            newlyConfirmed =
                orderRec.Status__c == 'Confirmed' &&
                oldOrder.Status__c != 'Confirmed';
        }

        if (newlyConfirmed && orderRec.Vehicle__c != null) {
            Integer count = quantityRequired.containsKey(orderRec.Vehicle__c)
                ? quantityRequired.get(orderRec.Vehicle__c) : 0;
            quantityRequired.put(orderRec.Vehicle__c, count + 1);
        }
    }

    List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

```

```

for (Id vehicleId : quantityRequired.keySet()) {
    Vehicle__c vehicle = vehicleStockMap.get(vehicleId);
    Integer required = quantityRequired.get(vehicleId);

    if (vehicle != null &&
        vehicle.Stock_Quantity__c != null &&
        vehicle.Stock_Quantity__c >= required) {
        vehicle.Stock_Quantity__c -= required;
        vehiclesToUpdate.add(vehicle);
    }
}

if (!vehiclesToUpdate.isEmpty()) {
    update vehiclesToUpdate;
}
}
}

```

6.2 VehicleOrderTrigger

```

trigger VehicleOrderTrigger on Vehicle_Order__c (
    before insert,
    before update,
    after insert,
    after update
){
    VehicleOrderTriggerHandler.handleTrigger(
        Trigger.new,
        Trigger.oldMap,
        Trigger.isBefore,
        Trigger.isAfter,
        Trigger.isInsert,
        Trigger.isUpdate
    );
}

```

7 Asynchronous Apex - Batch and Scheduler

Batch Apex was created to process Vehicle Order records that remain Pending. The batch checks the related vehicle stock and changes eligible orders to Confirmed. The existing VehicleOrderTrigger then applies the stock-reduction logic when the order status changes to Confirmed.

7.1 VehicleOrderBatch

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {
```

```
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
            AND Vehicle__c != null
        ]);
    }
}
```

```
    global void execute(
        Database.BatchableContext bc,
        List<Vehicle_Order__c> orderList
    ) {
        Set<Id> vehicleIds = new Set<Id>();
```

```
        for (Vehicle_Order__c orderRec : orderList) {
            vehicleIds.add(orderRec.Vehicle__c);
        }
```

```
        Map<Id, Vehicle__c> vehicleStockMap =
            new Map<Id, Vehicle__c>([
                SELECT Id, Stock_Quantity__c
                FROM Vehicle__c
                WHERE Id IN :vehicleIds
                FOR UPDATE
            ]);
```

```
        List<Vehicle_Order__c> ordersToUpdate =
            new List<Vehicle_Order__c>();
```

```
        Map<Id, Integer> remainingStock = new Map<Id, Integer>();
```

```
        for (Vehicle__c vehicle : vehicleStockMap.values()) {
            Integer qty = vehicle.Stock_Quantity__c == null
                ? 0
                : Integer.valueOf(vehicle.Stock_Quantity__c);
            remainingStock.put(vehicle.Id, qty);
        }
```

```
        for (Vehicle_Order__c orderRec : orderList) {
            Integer stock = remainingStock.get(orderRec.Vehicle__c);
```

```
            if (stock != null && stock > 0) {
                orderRec.Status__c = 'Confirmed';
                ordersToUpdate.add(orderRec);
```

```
            remainingStock.put(
```

```

        orderRec.Vehicle__c,
        stock - 1
    );
}
}

if (!ordersToUpdate.isEmpty()) {
    update ordersToUpdate;
}
}

global void finish(Database.BatchableContext bc) {
    System.debug('Vehicle order batch job completed.');
```

7.2 VehicleOrderBatchScheduler

```

global class VehicleOrderBatchScheduler implements Schedulable {

    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50);
    }
}
```

7.3 Daily Scheduling

The schedulable class was registered through Execute Anonymous using a midnight CRON expression:

```

String cronExp = '0 0 0 * * ?';

System.schedule(
    'Daily Vehicle Order Processing',
    cronExp,
    new VehicleOrderBatchScheduler()
);
```

The Scheduled Jobs page confirmed that Daily Vehicle Order Processing was registered as Scheduled Apex with the next scheduled run at 12:00 AM.

https://orgfarm-3ee0fd429-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ScheduledJobs/home

Developer Edition

Setup Home Object Manager

Scheduled Jobs

All Scheduled Jobs

The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.

Percentage of Scheduled Jobs Used: 1%
You have currently used 1 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: All Scheduled Jobs Create New View

Schedule Apex							
Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Vehicle Order Processing	Mishra, Rithik	9/10/2026, 8:39 AM		9/11/2026, 12:00 AM	Scheduled Apex	08eg700000I79HL
Del	MetaYtics Data Loader Job for Org : 00Dg700000F8xK1	User Integration	9/4/2026, 2:01 PM	9/10/2026, 6:12 AM	9/11/2026, 6:12 AM	Autonomous Data Loader Job	08eg700000HET2G
	Program Milestone Computation Cron Job	Process, Automated	9/4/2026, 2:01 PM	9/10/2026, 7:00 AM	9/10/2026, 11:59 AM	Program Milestone Computation Cron Job	08eg700000HET2E
	Program Status Update Cron Job	Process, Automated	9/4/2026, 2:01 PM	9/10/2026, 5:01 AM	9/10/2026, 8:00 PM	Program Status Update Cron Job	08eg700000HET2F

Figure 7: Daily Vehicle Order Processing listed under Scheduled Jobs.

8 Implementation Testing and Evidence

The completed implementation was verified using UI-based functional tests. The following table records the evidence observed during the project work.

Component	Observed Test / Evidence	Result
Custom Objects and Tabs	Six custom-object tabs visible in Salesforce Setup.	Completed
Auto Assign Dealer Flow	Flow saved and activated with customer retrieval, dealer lookup, and order update elements.	Activated
Test Drive Reminder	Created TD-0001 with Scheduled status and received the reminder email.	Passed
Out-of-Stock Trigger	Attempted Confirmed order for a vehicle with Stock Quantity = 0.	Passed - save blocked with custom error
Confirmed Order / Stock	Confirmed order created after stock became available; vehicle record reflected remaining stock.	Passed
Scheduled Apex	Daily Vehicle Order Processing visible in Scheduled Jobs with next run at 12:00 AM.	Passed - schedule registered
Batch Pending-to-Confirmed	Batch class created and scheduled; a separate manual transition test was not captured in the screenshots used for this report.	Configured

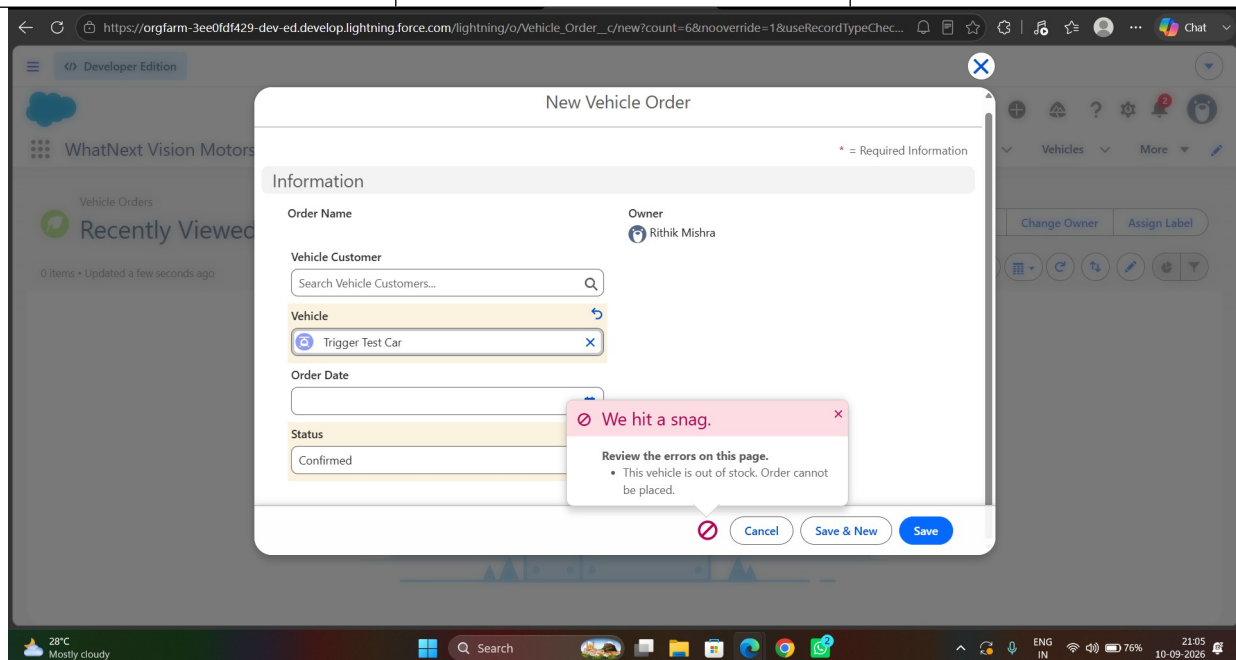


Figure 8: Out-of-stock validation blocks a Confirmed Vehicle Order.

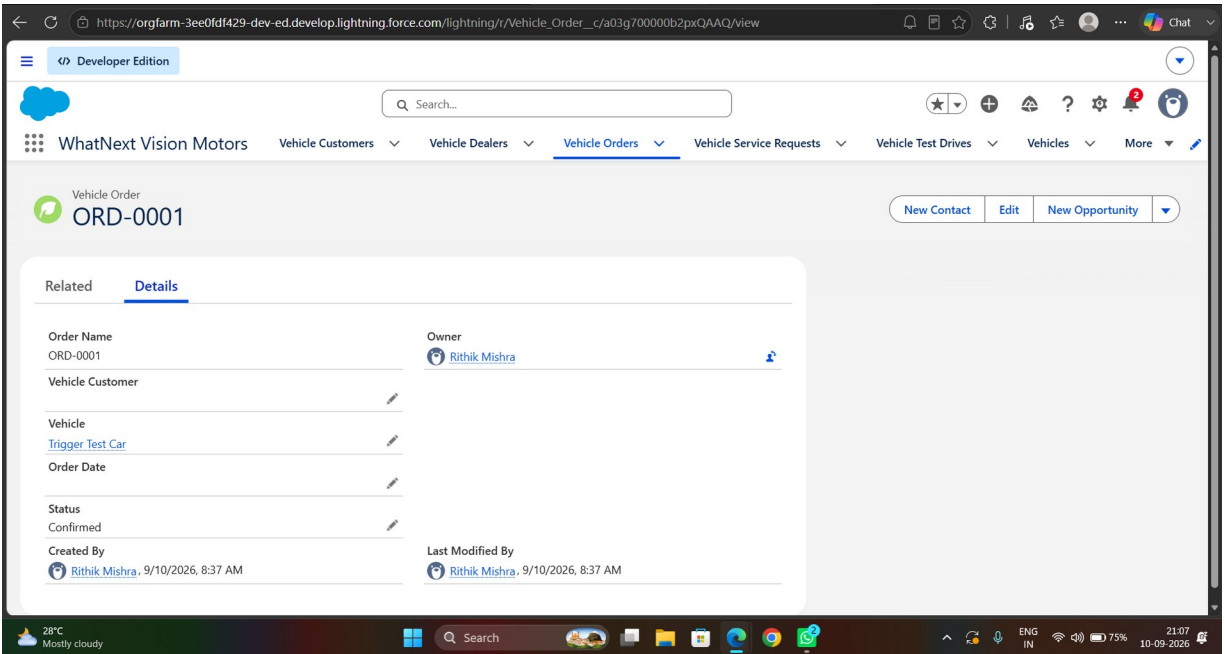


Figure 9: Confirmed Vehicle Order created after stock was made available.

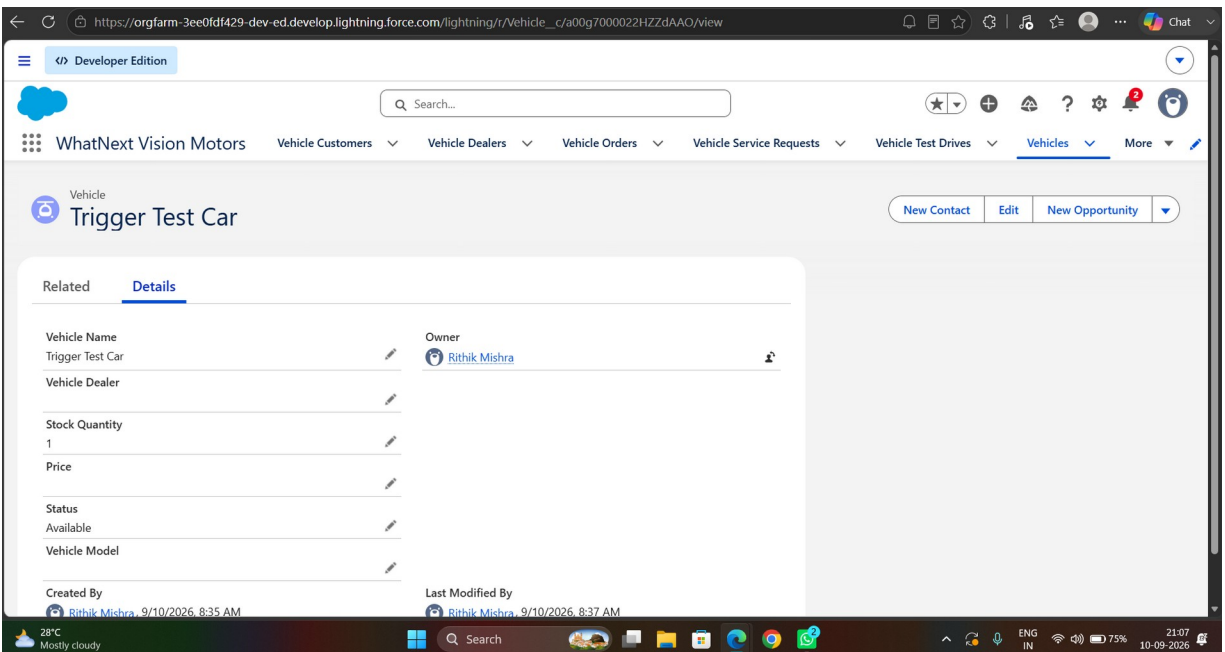


Figure 10: Vehicle record showing remaining Stock Quantity after order processing.

9 Final Implemented Architecture

The implementation can be summarized as an end-to-end Salesforce automation pipeline:

- Custom Objects and Lookup Relationships store the automotive CRM data.
- The WhatNext Vision Motors Lightning app exposes all primary custom-object tabs.
- Auto Assign Dealer runs for newly created Pending Vehicle Orders and populates the Dealer lookup using location matching.
- Test Drive Reminder runs on a scheduled path one day before the Test Drive Date and sends an email to the related customer.
- VehicleOrderTrigger and VehicleOrderTriggerHandler enforce stock validation and update stock for confirmed orders.
- VehicleOrderBatch processes Pending orders in bulk when stock is available.
- VehicleOrderBatchScheduler launches the batch with a batch size of 50.
- Daily Vehicle Order Processing is registered as Scheduled Apex for 12:00 AM.

9.1 Logical Flow

CUSTOM OBJECTS

```
|
+--> Vehicle Order --(Pending)--> Auto Assign Dealer Flow
|
|           |
|           +--> Customer Address
|           +--> Matching Dealer Location
|           +--> Update Order.Dealer
|
+--> Vehicle Test Drive --(Scheduled)--> Scheduled Path (1 day before)
|
|           |
|           +--> Get Customer
|           +--> Send Reminder Email
|
+--> Vehicle Order DML --> VehicleOrderTrigger
|
|           +--> Block Confirmed order if stock <= 0
|           +--> Decrease stock when order becomes Confirmed
```

MIDNIGHT SCHEDULE

```
|
+--> VehicleOrderBatchScheduler
|
|   +--> VehicleOrderBatch
|       |
|       +--> Query Pending Orders
|       +--> Confirm eligible orders when stock is available
```

10 Conclusion

The completed WhatNext Vision Motors Salesforce implementation delivers the core CRM configuration and automation required for vehicle sales and test-drive operations. Six custom objects, their business fields, Lookup Relationships, custom tabs, and a Lightning app were configured. Two Record-Triggered Flows were activated for dealer assignment and test-drive reminders.

Programmatic automation was added through a Trigger Handler and Apex Trigger to enforce stock rules, while Batch Apex and Scheduled Apex were configured for recurring processing of pending orders. Functional testing demonstrated successful out-of-stock blocking and successful customer email delivery, and the daily Scheduled Apex job was verified in Salesforce Setup.

This report intentionally documents only the work completed in the Salesforce org and avoids claiming additional features that were present in the reference report but were not implemented during this project session.